

**Istruzioni.** Durante lo svolgimento del test, attenersi rigorosamente alle seguenti istruzioni.

- Gli esercizi devono essere consegnati attraverso il sito Moodle, al seguente indirizzo:

<https://elearning.unica.it/course/view.php?id=188>

- Ogni esercizio deve essere consegnato su un file con estensione `.ml`. Non sono ammesse altre modalità di consegna.
- Non è possibile consegnare alcun esercizio dopo la scadenza stabilita. È possibile re-inviare ogni esercizio un numero arbitrario di volte entro la scadenza stabilita, senza subire penalizzazioni.
- Non è ammesso alcun tipo di collaborazione.
- Se viene richiesto che una funzione abbia un nome specifico, la funzione deve avere quel nome.
- Il file contenente la soluzione non deve contenere esempi (anche commentati).
- Nelle soluzioni degli esercizi 7 e 8 il tipo deve essere contenuto nel file.

1. Scrivere una funzione `foo` con il seguente tipo:

`foo: 'a -> 'a -> (int -> int) -> int`

2. Si consideri la successione  $(a_n, b_n)_{n \in \mathbb{N}}$  di coppie di numeri interi data da:

$$\begin{aligned} a_0 &= 0 & b_0 &= 1 \\ a_{n+1} &= \begin{cases} 2b_n + 1 & \text{se } b_n \text{ pari} \\ 2b_n & \text{altrimenti} \end{cases} & b_{n+1} &= \begin{cases} a_n + 1 & \text{se } n \text{ dispari} \\ a_{n+1} & \text{altrimenti} \end{cases} \end{aligned}$$

Scrivere una funzione `seq: int -> (int * int)` tale che l'espressione `seq n` valuta alla coppia  $(a_n, b_n)$ . Nota: è consentito definire funzioni ausiliarie.

3. Date due funzioni  $f, g: \mathbb{Z} \rightarrow \mathbb{Z}$ , si definisca la funzione  $f \circ g: \mathbb{Z} \rightarrow \mathbb{Z}$  come segue:

$$(f \circ g)(x) = \begin{cases} g(x) & \text{se } g(x) \text{ è multiplo di } x \\ |f(x) + g(x)| & \text{altrimenti} \end{cases}$$

Definire una funzione `comp` con il seguente tipo:

`comp: (int -> int) -> (int -> int) -> (int -> int)`

e tale che `comp f g = f ∘ g`.

4. Scrivere una funzione `foo` con il seguente tipo:

```
foo: 'a list -> 'b list -> 'a list * 'b list
```

5. Definire una funzione `fool` con tipo:

```
'a list -> int -> 'a -> 'a list
```

in modo che `fool l n v` sia la lista ottenuta da `l` lasciando inalterati i primi `n` elementi, e rimpiazzando i rimanenti con `v`. Ad esempio,

```
fool [1;2;3;4;5;6;7;8;9;10] 3 0 = [1;2;3;0;0;0;0;0;0;0]
```

6. Scrivere una funzione `foo` con il seguente tipo:

```
foo: ('a -> 'b) -> 'a list -> 'b * 'b
```

7. Si consideri il seguente tipo:

```
type 'a tree = Empty | Node of 'a * 'a tree * 'a tree
```

Definire una funzione `sumLeaf` con tipo:

```
'int tree -> int
```

in modo che `sumLeaf t` restituisca la somma dei valori delle foglie dell'albero `t`. Ad esempio:

```
# let t1 = Node (1, Node(2, Node( 3, Empty, Empty), Node(4, Empty, Empty)),
Empty);;
```

```
# sumLeaf t1;;
```

```
- : int = 7
```

8. Si consideri la seguente definizione di tipo:

```
type boolean = One
```

```
| Zero
```

```
| And of boolean * boolean
```

```
| Or of boolean * boolean
```

```
| Implies of boolean * boolean
```

```
| Equiv of boolean * boolean;;
```

Definire una funzione `eval` che assegni ad un espressione di tipo `boolean` un valore di tipo `bool`, interpretando `One` e `Zero` con i booleani `true` e `false`, `And`, `Or` con la congiunzione e la disgiunzione booleana, `Implies` con l'implicazione e `Equiv` con l'equivalenza. Si ricorda che l'implicazione "Se  $A$  allora  $B$ " equivale all'espressione booleana  $(\text{not } A) \text{ or } B$ .